```
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn.metrics import accuracy_score
```

```
loan_dataset = pd.read_csv('/content/loan_train.csv')
```

```
type(loan_dataset)
```

```
pandas.core.frame.DataFrame
def __init__(data=None, index: Axes | None=None, columns: Axes | None=None,
dtype: Dtype | None=None, copy: bool | None=None) -> None
```

/usr/local/lib/python3.12/dist-packages/pandas/core/frame.py

```
Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Data structure also contains labeled axes (rows and columns).
Arithmetic operations align on both row and column labels. Can be
thought of as a dict-like container for Series objects. The primary
```

```
loan_dataset.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantInc
0	LP001002	Male	No	0	Graduate	No	5
1	LP001003	Male	Yes	1	Graduate	No	4
2	LP001005	Male	Yes	0	Graduate	Yes	3
3	LP001006	Male	Yes	0	Not Graduate	No	2
4	LP001008	Male	No	0	Graduate	No	6

Next steps: Generate code with loan_dataset New interactive sheet

```
loan_dataset.isnull().sum()
```

	0
Loan_ID	0
Gender	13
Married	3
Dependents	15
Education	0
Self_Employed	32
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	22
Loan_Amount_Term	14
Credit_History	50
Property_Area	0
Loan_Status	0

dtype: int64

```
loan_dataset.shape
```

```
(614, 13)
```

```
loan_dataset.describe()
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_
count	614.000000	614.000000	592.000000	600.00000	56
mean	5403.459283	1621.245798	146.412162	342.00000	
std	6109.041673	2926.248369	85.587325	65.12041	
min	150.000000	0.000000	9.000000	12.00000	
25%	2877.500000	0.000000	100.000000	360.00000	
50%	3812.500000	1188.500000	128.000000	360.00000	
75%	5795.000000	2297.250000	168.000000	360.00000	
max	81000.000000	41667.000000	700.000000	480.00000	

```
loan_dataset = loan_dataset.dropna()
```

```
loan_dataset.isnull().sum()
```

	0
Loan_ID	0
Gender	0
Married	0
Dependents	0
Education	0
Self_Employed	0
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	0
Loan_Amount_Term	0
Credit_History	0
Property_Area	0
Loan_Status	0

dtype: int64

```
loan_dataset.shape
```

```
(480, 13)
```

1. Y = 1
2. N = 0

```
loan_dataset.replace({"Loan_Status" :{'N':0, 'Y':1}},inplace=True)
```

```
/tmp/ipykernel_13499/1371842469.py:1: FutureWarning: Downcasting behavior in `re
  loan_dataset.replace({"Loan_Status" :{'N':0, 'Y':1}},inplace=True)
```

```
loan_dataset.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantInc
1	LP001003	Male	Yes	1	Graduate	No	4
2	LP001005	Male	Yes	0	Graduate	Yes	3
3	LP001006	Male	Yes	0	Not Graduate	No	2
4	LP001008	Male	No	0	Graduate	No	6
5	LP001011	Male	Yes	2	Graduate	Yes	5

Next steps: Generate code with `loan_dataset` New interactive sheet

```
# Depedent Column Values
loan_dataset['Dependents'].value_counts()
```

Dependents	count
0	274
2	85
1	80
3+	41

dtype: int64

```
# Replacing the value of 3+ to 4

loan_dataset = loan_dataset.replace(to_replace='3+', value=4)
```

```
loan_dataset['Dependents'].value_counts()
```

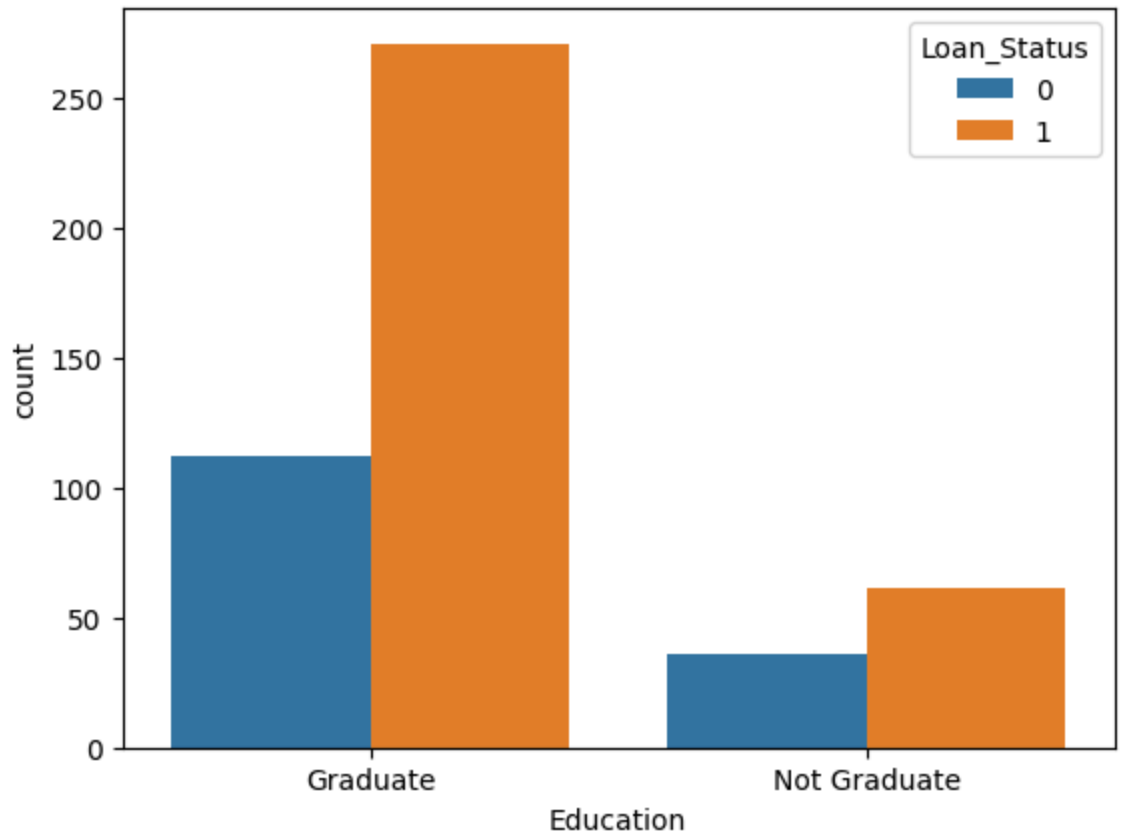
Dependents	count
0	274
2	85
1	80
4	41

dtype: int64

Data Visualization

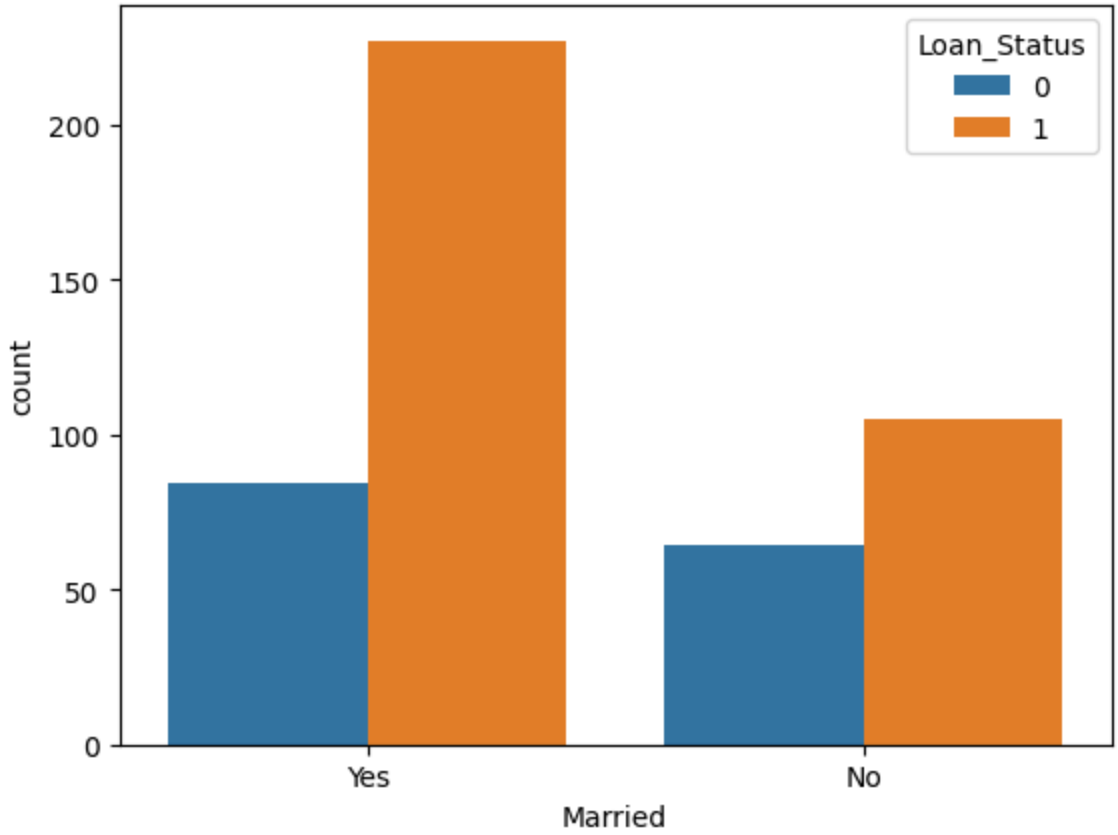
```
# education and loan status
sns.countplot(x='Education',hue='Loan_Status',data=loan_dataset)
```

```
<Axes: xlabel='Education', ylabel='count'>
```



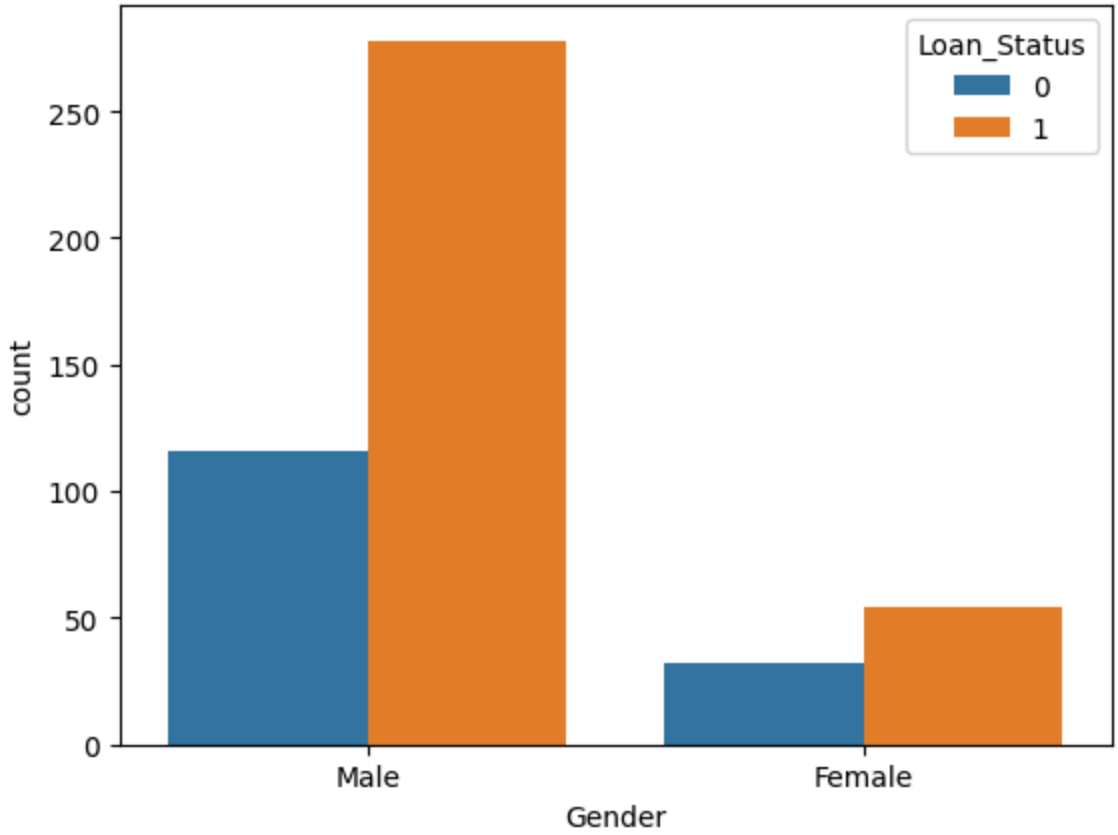

```
# married and loan status
sns.countplot(x='Married',hue='Loan_Status',data=loan_dataset)
```

```
<Axes: xlabel='Married', ylabel='count'>
```



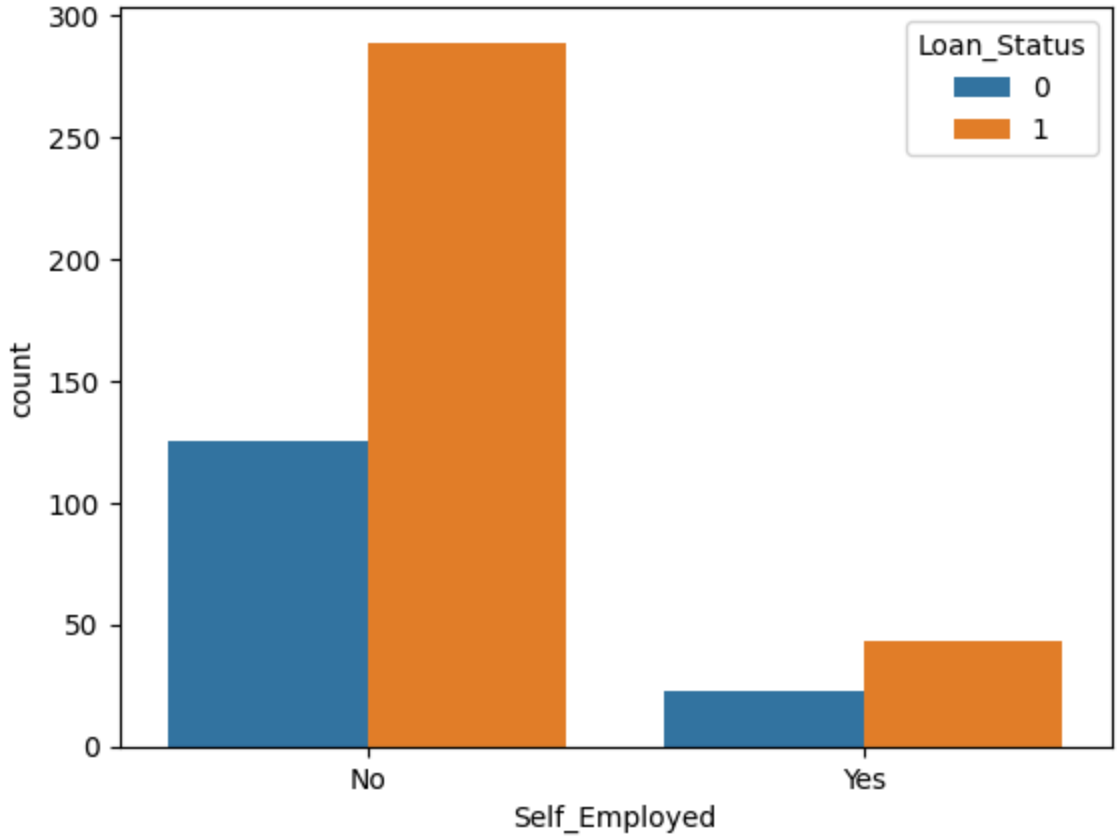

```
# gender and loan status
sns.countplot(x='Gender',hue='Loan_Status',data=loan_dataset)
```

```
<Axes: xlabel='Gender', ylabel='count'>
```



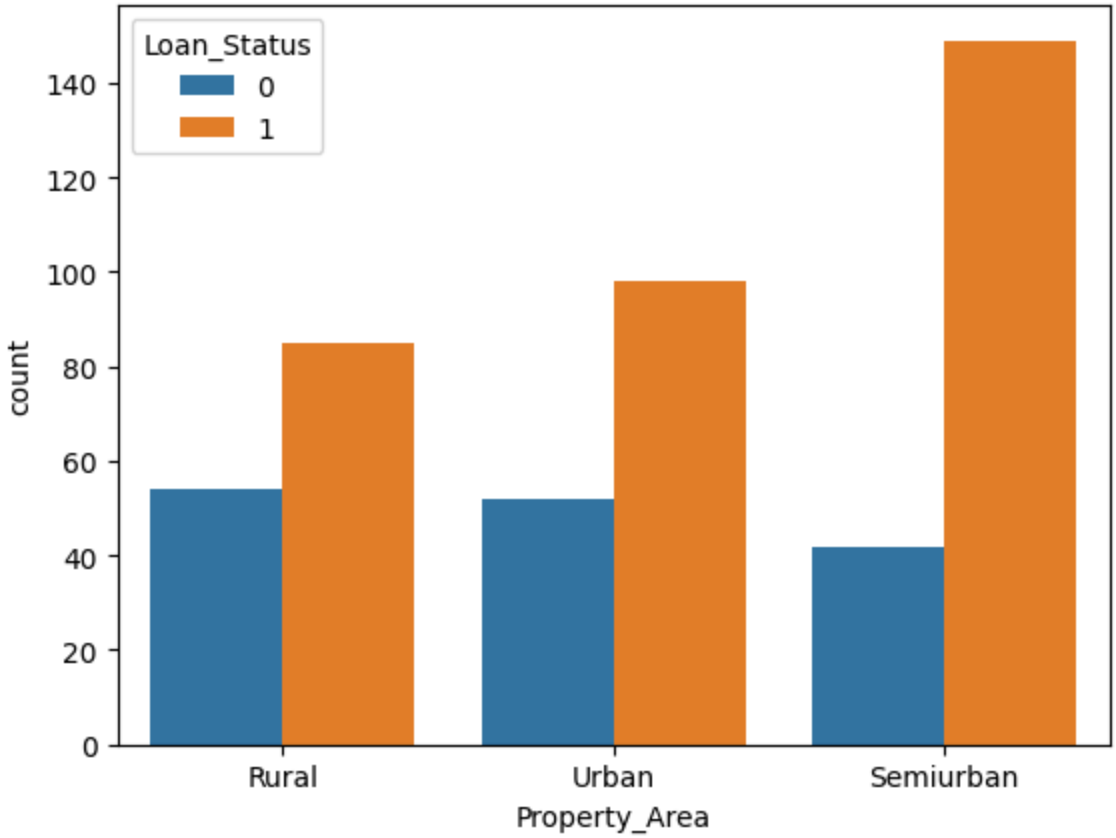

```
# employement and loan status
sns.countplot(x='Self_Employed',hue='Loan_Status',data=loan_dataset)
```

```
<Axes: xlabel='Self_Employed', ylabel='count'>
```




```
# property area and loan status
sns.countplot(x='Property_Area',hue='Loan_Status',data=loan_dataset)
```

```
<Axes: xlabel='Property_Area', ylabel='count'>
```



Married

1. No = 0
2. Yes = 1

Gender

1. Male = 1
2. Female = 0

Self_Employed

1. Yes = 1
2. No = 0

Property_Area

1. Rural = 0
2. Semiurban = 1
3. Urban = 2

Education

1. Graduate = 1
2. Not Graduate = 0

```
# convert categorical columns to numerical values

loan_dataset.replace({"Married" :{'No':0, 'Yes':1}, 'Gender':{'Male':1, 'Female
                      'Property_Area':{'Rural':0, 'Semiurban':1, 'Urban':2}, 'E
```

```
/tmp/ipykernel_13499/1577032734.py:3: FutureWarning: Downcasting behavior in `re
  loan_dataset.replace({"Married" :{'No':0, 'Yes':1}, 'Gender':{'Male':1, 'Femal
```

```
loan_dataset.head()
```

Next steps: Generate code with `loan_dataset` New interactive sheet

```
loan_dataset.to_csv('loan_dataset.csv')
```

```
X = loan_dataset.drop(columns=['Loan_ID','Loan_Status'],axis=1)
```

```
Y = loan_dataset['Loan_Status']
```

```
X.head(10)
```

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Coap
1	1	1	1	1	0	4583	
2	1	1	0	1	1	3000	
3	1	1	0	0	0	2583	
4	1	0	0	1	0	6000	
5	1	1	2	1	1	5417	
6	1	1	0	0	0	2333	
7	1	1	4	1	0	3036	
9	1	1	1	1	0	12841	
10	1	1	2	1	0	3200	

Next steps: Generate code with X New interactive sheet

```
Y.head(10)
```

	Loan_Status
1	0
2	1
3	1
4	1
5	1
6	1
7	0
8	1
9	0
10	1